RealMLP

RealMLP est une version de MLP plus moderne. Testez-la avec le jeu de donnée du cours passé :

```
task = openml.tasks.get_task(361113)
dataset = openml.datasets.get_dataset(task.dataset_id, download_data=False)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, train_size=15000, random_state=0
)
X_test = X_test[:10000]
y_test = y_test[:10000]
```

Pour le code, utilisez la version originale de RealMLP, disponible ici : https://github.com/dholzmueller/realmlp-td-s_standalone

Essayez d'entraîner (fit) et de prédire (predict). Sur Colab (GPU T4), testez les options cpu et cuda. Ensuite, testez d'autres modèles de scikit-learn (dont MLPClassifier lui-même) et comparez les résultats.

Fonctions d'activation

Faire un dessin (à l'aide de `matplotlib`) de la fonction d'activation `nn.SELU` utilisée dans RealMLP.

Ensuite, faire de même pour d'autres fonctions d'activation : https://docs.pytorch.org/docs/main/nn.html#non-linear-activations-weighted-sum-nonlinearity (des fonctions utilisées souvent sont par exemple `ELU`, `ReLU`, `LeakyReLU`, `GELU`, `Sigmoid`, `Tanh`).

Régression logistique

Essayer la régression logistique vue au cours sur le même jeu de données choisi dans l'autre exercice pour RealMLP.

Autres classificateurs

Utiliser d'autres classificateurs scikit-learn sur MNIST (e.g. TreeClassifier, RandomForestClassifier, NearestNeighbors, LinearSVC, SVC). Quels résultats peut-on obtenir ?

Autres fonctions de perte

Implémenter autres fonctions de perte, par exemple la distance L1 ou L2 entre les prédictions (après softmax) et labels. Le faire dans la version NumPy et après dans la version PyTorch (où la fonction de perte L2 est implémentée par la classe `MSELoss`).

Normalisation

Répéter entraînement et évaluation de la version NumPy sans diviser par 255.0. Le résultat est-il différent ? Expliquer pourquoi on pouvait imaginer la réponse sans lancer l'expérience numérique.

nn.Linear à la main

Implémenter à la main nn.Linear, en définissant les poids dans `__init__` et le produit scalaire + biais dans `forward`. À noter :

- Les paramètres doivent être définis avec `nn.Parameter` ; par exemple, similaire à `self.A = nn.Parameter(torch.zeros(dim1, dim2, ...))` (mais probablement initialisés avec des valeurs aléatoires, pas zéro). Sinon, ils seraient des constantes et pas mis à jour par `backward`.
- Le produit de tenseurs en PyTorch est appelé par `torch.matmul` ou bien avec `@`.

SGD

Créer une classe SGD qui fait le même que `torch.optim.SGD`, et vérifier qu'il marche en remplaçant `torch.optim.SGD` par cette fonction dans l'entraînement de la régression logistique. Ceci doit avoir trois méthodes :

- `__init__(self, params, lr)` qui mémorise ces paramètres dans des attributes
- `step(self, *args, **kwargs)` qui pour tout paramètre `param` mémorisé, modifie `param.data` en soustrayant `param.grad * lr`
- `zero_grad(self, *args, **kwargs)` qui pour tout paramètre `param` appelle `.grad.zero_()` comme ci-dessus.

Réseaux de neurones

Temperature

Dessiner les fonctions $f(x) = 1/(1 + e^{-\tau x})$ pour différentes valeurs de $\tau > 0$. (Le paramètre τ est parfois appelé *temperature*). Que se passe-t-il quand $\tau \to \infty$?

Optimisateur

Dans l'implémentation PyTorch MLP, essayer de remplacer Adam par SGD et jouer avec les paramètres pour essayer d'obtenir une convergence à peu près.

ModuleList

Modifier la définition de `TorchMLP` avec un nombre arbitraire de couches en utilisant `nn.ModuleList` à la place de `nn.Sequential`. La différence est qu'avec `nn.ModuleList` tous les modules doivent être appelés séparément dans `forward`.

Fonction de perte

Comme dans la régression logistique, essayer de remplacer la fonction de perte par L1 ou L2. Essayer également avec la somme de l'une d'entre elles et l'entropie croisée.

Gradients

PyTorch permet de calculer des gradients (dans des points donnés, mais de façon analytique) ; par exemple, pour calculer la dérivée de la fonction $f(x) = x^2$ dans le point $x = 3$ on peut faire :

```
xt = torch.tensor(5.0, requires_grad=True)
yt = xt ** 2
yt.backward()
print(xt.grad)  # donne 10.0
```

Question 1

Après avoir fait tourner le code ci-dessus, essayez de calculer `yt = xt ** 3` suivi encore par `yt.backward()` et `print(xt.grad)`. Quelle est la dérivée de $f(x) = x^3$? A-t-on le bon résultat dans $x = 5$ (c'est-à-dire, `75.0`) ?

Question 2

(normalement, la réponse ci-dessus est non). La raison est que, par défaut, PyTorch *somme tous les gradients* sur la même variable. Donc normalement ci-dessus on obtient `10.0 + 75.0 = 85.0`. C'est pour ça que, dans les entraînements, on a `optimizer.zero_grad()` à chaque tour. Pour faire de même dans une seule variable, on utilise plutôt `xt.grad.zero_()`. Vérifier qu'en appelant ça le calcul se réinitialise et permet de calculer les bonnes valeurs.

Gradients en parallèle

PyTorch est surtout bon pour des gros calculs en parallèle. On suppose maintenant de vouloir dessiner la fonction $f(x) = x \sin(\frac{1}{x})$ entre 10^{-5} et 0.1 et tracer également une droite tangente dans les points $0.01, 0.02, \ldots, 0.1$.

Graphe

Tracer le graphe de cette fonction avec numpy et matplotlib.

Dérivées

Calculer les 10 dérivées dans les points demandés en utilisant l'approche ci-dessus de PyTorch (dans une boucle)

Graphe des dérivées

Dessiner des traits de droites qui passent par les points $(x_i, f(x_i))$ et ont pente égale à la dérivée qui vient d'être calculée, pour $x_1 = \frac{1}{100}, \ldots, x_{10} = \frac{10}{100}$.

Dérivées en parallèle

Mais on peut calculer les 10 dérivées dans une seule étape, en partant de `xt = torch.tensor([0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1], requires_grad=True)`. Malheureusement on ne peut pas juste faire `yt = xt * torch.sin(xt ** -1)` suivi par `yt.backward()`, car le calcul de `yt` est le bon mais `backward()` n'accepte que des fonctions

scalaires (c'est-à-dire, dont l'output a dimension 1). Mais presque... comment transformer ce tenseur, qui serait le vecteur $(f(x_1), \dots, f(x_{10}))$ dans une fonction $F(x_1, \dots, x_{10}) : \mathbb{R}^{10} \to \mathbb{R}$, telle que les 10 dérivées partielles $\frac{\partial F}{\partial x_1}, \dots, \frac{\partial F}{\partial x_{10}}$ soient égales aux 10 dérivées qu'on souhaite $\frac{df}{dx}(x_1), \dots, \frac{df}{dx}(x_{10})$?

Trouver un bon taux d'apprentissage

On va implémenter une version simplifiée de lr_find par ici : https://sgugger.github.io/how-do-you-find-a-good-learning-rate.html qui aide à chercher un bon taux d'apprentissage.

Le choix du taux d'apprentissage est crucial pour une convergence rapide et efficace. Dans la suite on décrit une façon semi-automatique de trouver un taux acceptable.

Taux d'apprentissage varié dans l'entraînement

En utilisant un des modèles MLP de PyTorch implémentés dans le cours, modifier le code d'entraînement pour effectuer 100 pas d'apprentissage, en utilisant à chaque pas un taux d'apprentissage entre 10^{-8} et 10, espacié régulièrement de *façon géométrique*, c'est-à-dire $\varepsilon_0 = 10^{-8}, \varepsilon_1 = \alpha \cdot 10^{-8}, \varepsilon_2 = \alpha^2 \cdot 10^{-8}, \dots, \varepsilon_{99} = \alpha^{99} \cdot 10^{-8} = 10$ (trouver le bon α pour que cela soit correct).

Sauvegarde

À chaque étape d'entraînement, sauvegarder dans une liste le taux d'apprentissage ε_i ainsi que la perte calculée dans cette étape.

Graphe

Tracer le graphe (en utilisant une échelle logarithmique) du rapport entre taux d'apprentissage et perte. Limiter l'axe des y entre, par exemple, la moitié de la valeur minimum et 5 fois cette valeur.

Valeur "optimale"

De façon assez heuristique, on choisit comme "bon" taux ε une valeur environ 10 fois plus petite que le point de minimum de cette fonction (c'est-à-dire, si le minimum est dans le point $\varepsilon_{60} = \alpha^{60} \cdot 10^{-8} \approx 0.00285$, on choisit $\varepsilon \approx 0.000285$; comme ce n'est qu'une heuristique, dans la pratique on choisirait probablement 0.03. Écrire le code qui imprime automatiquement la valeur où le graphe ci-dessus atteint un minimum et la valeur "optimale" 10 fois plus petite, et trace cela dans un graphe. Ci-dessous le résultat souhaité (ici obtenu pour la version "base" de MLP avec une couche cachée de 100 neurones et optimisateur SGD). Remarque : un bon réseau de neurones aurait une courbe plus "douce" à la gauche du minimum, de façon à avoir moins de différence dans la fonction de perte entre le minimum et la valeur choisie. Ici nous utilisons un MLP assez grossier, donc on perd quelque chose avec cette heuristique. En tout cas, en choisissant un taux d'apprentissage 10 fois plus petit que le minimum, au pire on va rendre l'entraînement 10 fois plus lent (mais probablement beaucoup moins que 10 fois) ; en choisissant un taux d'apprentissage trop élevé, on risque de ne rien apprendre. Essayez le même dessin avec Adam ou d'autres réseaux.

Approximation d'une fonction

Comme exemple jouet, on va créer un réseau de neurones qui s'approche de la fonction $y = \sin(x)$ sur un domaine fixé $[0, 10]$. On utilisera le modèle `TorchMLP` avec un nombre arbitraire de couches et la fonction de perte `MSELoss` (c'est de la régression, non pas de la classification !). L'input x et l'output y vivent dans $\mathbb{R}$ (en dimension 1). On commence par un modèle avec une seule couche intermediaire, de 10 neurones.

Boucle d'entraînement

Implémenter la boucle d'entraînement comme dans le cours (quelque milliers de pas), avec les différences suivantes :

- les données `torch_X_train` sont engendrés de façon aléatoire à chaque tour, par exemple 1000 points selon la loi uniforme $U([0, 10])$.
- les données `torch_y_train` en sont déduites par `torch.sin`
- pour vérifier l'avancement de l'apprentissage, imprimer directement la valeur calculée par la fonction de perte (il n'y a pas vraiment de distinction train / test, ni de fonction de précision...)

Visualisation

Crée un tenseur qui a 1000 points régulièrement espaciés sur `[0, 10]` et dessiner, à l'aide de `matplotlib`, la vraie fonction $\sin(x)$ et les valeurs estimées par le modèle. Que se passe-t-il si on étend le domaine un peu, par exemple sur $[-3, 13]$?

Autres modèles

Essayer d'augmenter le nombre et / ou la taille des couches intermediaires. Comment évolue la courbe prédite ? Et si on remplace la fonction d'activation par d'autres fonctions, telle que GELU ou Sigmoid ?

Autre fonction de perte

On peut "faire semblant" que ça soit un modèle de classification, en prenant les deux classes comme valeurs -1 et +1 ; on interprète $y \in [-1, 1]$ comme la probabilité $(y+1)/2$ d'être dans la deuxième classe. Modifier le code pour suivre cette interprétation : pour cela, on utilise la fonction de perte `torch.nn.BCEWithLogitsLoss`, et on transforme `torch_y_train` comme décrit. Pour dessiner le résultat, il faudra également appliquer la transformation inverse `torch.nn.Sigmoid()(y_pred) * 2 - 1`.

Dataset, DataLoader

On reprend ici le code vu au cours dans la section "Dataset, DataLoader".

Taille du lot

Essayer avec différentes tailles de lots (`batch_size`). Entraîner toujours pour le même nombre d'époques, et voir s'il y a des différences au niveau de la précision du modèle obtenu, et du temps d'entraînement (pour cela, on peut utiliser la fonction `trange` de la bibliothèque `tqdm` à la place du `range` extérieur, par exemple)

Importance du shuffle

Dans la pratique, c'est important que les données soient mélangées à chaque époque ; ou du moins, qu'elles soient mélangées une fois. Parfois, les données arrivent triées d'une façon spécifique, ce qui peut endommager l'apprentissage, si on oublie de les mélanger. On va ici simuler ce problème en triant le jeu de données, pour voir que les résultats deviennent moins bons.

- Trier `idx_train` de façon à ce que `np.array(dataset.labels)[idx_train]` soit croissant.
- Définir à nouveau `train_dataset` en utilisant ces nouveaux indices
- Modifier le paramètre `shuffle` dans `train_dataloader`.
- Relancer l'entraînement ci-dessus, en utilisant une taille des lots pas trop grande. Normalement, la qualité obtenue va être beaucoup moins bonne.

Data augmentation

Surtout quand on utilise des jeux de données petits (comme MNIST), et notamment dans le traitement d'images, on utilise presque toujours de *l'augmentation des données*. C'est une technique pour simuler qu'on a plus de données qu'on n'a vraiment, en les modifiant légèrement avec des transformations contrôlées, qui ne changent pas l'étiquette à prédire. Dans le cas d'images naturelles (photos, par exemple), on peut par exemple modifier l'exposition, le contraste, ou prendre la symétrie miroir de l'image. Dans le cas de MNIST, où les nombres sont toujours en blanc pur sur noir pur, changer les couleurs n'a pas de sens ; la symétrie non plus, car on ne veut pas risquer de changer un 2 en 5 ou un 6 en 9. Mais il y a beaucoup de transformations qu'on peut faire, et qui aident l'apprentissage ; on va le faire ici.

Shift

On va ajouter un shift aléatoire d'un pixel ou deux. Dans le corps de `__getitem__`, d'abord modifier la taille du tenseur `image` de `784` à `(28, 28)`. Puis, implémenter une méthode `random_shift` de façon à pouvoir appeler :

```
image = self.random_shift(image)
```

qui, donnée un tenseur de taille `(28, 28)` :

- tire deux entiers `shift_x` et `shift_y` de façon aléatoire entre -2 et 2.
- retourne un autre tenseur de la même taille, mais où les pixels sont déplacés le long de x et y du nombre de pixels indiqué.

Finalement, à nouveau dans `__getitem__`, modifier à nouveau la taille de l'`image` obtenue pour la ramener à `784`.

Rotation

Sans doute on ne peut pas tourner l'image de 180 degrés, quitte à transformer un 6 en 9, mais quelques degrés peuvent aider. Il convient pour cela d'utiliser le paquet `cv2` (si vous ne l'avez pas, pour l'installer on utilise `pip install opencv-python`). Par exemple, le code pour tourner une image de 30 degrés est :

```
M = cv2.getRotationMatrix2D((14, 14),-30,1)
rotate_30 = cv2.warpAffine(image.numpy(),M,(28,28))
```

Ici, par contre, `image` doit être un array numpy. Écrire une méthode `random_rotation` qui tire un nombre aléatoire (entier ou float, peu importe) entre -10 et 10, transforme un tenseur de taille `(28, 28)` dans un array numpy de la même taille, le tourne, et retourne le tenseur PyTorch résultant.

Étirement

On ne doit pas se limiter à des transformations qui préservent la taille parfaitement. On peut par exemple étirer ou rétrécir le long du x ou y jusqu'au 20 %. Dans la transformation de rotation ci-dessus, `M` est une matrice de taille `(2, 3)` de la forme :

$$\begin{pmatrix} \cos(\theta) & \sin(\theta) & a \\ -\sin(\theta) & \cos(\theta) & b \end{pmatrix}$$

où a et b sont choisis de la bonne façon pour que la rotation soit autour du centre de l'image. Pour étirer l'image, on peut définir une matrice de la forme

$$\begin{pmatrix} c_x & 0 & 14(1-c_x) \\ 0 & c_y & 14(1-c_y) \end{pmatrix}.$$

Le résultat sera alors un étirement du facteur `c_x` le long de la largeur et `c_y` le long de la longueur. Implémenter une méthode `random_stretch` qui tire `c_x` et `c_y` de façon aléatoire entre `0.8` et `1.2` et applique l'étirement désiré.

Test

Une fois les trois méthodes ci-dessus implémentées, tirer des images du jeu de données résultant, et vérifier (avec `matplotlib`) que les images qu'on obtient soient bien compréhensibles par un être humain. Ensuite, commenter à rotation 2 sur les 3 méthodes implémentées et tirer des autres images, pour vérifier que chaque méthode fasse bien ce qu'on s'attend d'elle (déplacer, tourner, étirer). Astuce : pour ce deuxième test, on peut utiliser directement `dataset[0]` ou bien `dataset[1]` etc. pour être sûrs qu'on utilise toujours la même image de base, en tirant ainsi plusieurs versions de la même image modifiées par différents nombres aléatoires. De cette façon on peut être sûr, par exemple, que les augmentations font vraiment quelque chose, et ne se limitent pas à rendre l'image non modifiée.

Entraînement

Utiliser ce nouveau jeu de données pour entraîner un réseau de neurones. Voir si cela aide l'apprentissage. En particulier, tracer le graphe de la fonction de perte et de l'accuracy au fil de l'entraînement. Il peut convenir d'incrémenter le nombre d'époques d'apprentissage (car maintenant à chaque époque, le réseau voit une nouvelle image !)

Comparons avec la version sans augmentation

Est-ce que cela change le risque d'overfit du modèle ?

Traitement d'images, convolutions

Data augmentation, MNIST

Les mêmes transformations utilisées pour des MLP peuvent être appliquées aux CNN. Des 3 qu'on a vu, quelle(s) vont être utiles ? Quelle(s) sont assez inutiles ? [optionnel] : Essayez de les implémenter pour valider l'intuition.

Data augmentation, ImageNette

Le jeu de données ImageNette est quand même assez petit, il va profiter d'un peu d'augmentation. Implémenter quelques-uns des suivants :

- Rotation, comme ci-dessus
- À la place de l'étirement, on va plutôt choisir de façon aléatoire la partie d'image à couper : non pas toujours le carré central 160×160, mais un carré quelconque (par exemple, de taille minimum 120×120), ou même un rectangle (par exemple, de taille minimum 120 de chaque côté) qu'on va ensuite transformer en 160×160 par `image.resize((160, 160))`.
- Symétrie de 180 degrés
- Modification de luminosité, contraste, netteté
- Couleurs inversées, normalisation du contraste, égalisation, solarisation...

(toutes modifications implémentées dans PIL)